

CARTETHYIA: Master Architecture & Implementation Document

1. Project Overview & Identity

Name: Cartethyia **Theme:** Premium, Wuthering Waves-inspired Anime Social & Action-RPG Bot. **Core Philosophy:** Social interactions (/hug , /pat , chatting) are the primary resource generators for a deeply complex, turn-based Action-RPG endgame. **Target UX:** Expensive, polished, minimal text walls. Heavy use of Discord UI components (Buttons, Select Menus, Canvas-generated image profiles), and dynamic embed colors that match high-quality anime GIFs.

1.1 Technology Stack

- **Language:** TypeScript (Node.js)
- **Library:** Discord.js (v14+)
- **Database:** PostgreSQL managed via Prisma ORM
- **AI Engine:** Local LLM via Ollama API (`http://localhost:11434/api/generate`), utilizing `p-queue` for concurrency management.
- **Image Generation:** `canvas` or `@napi-rs/canvas` for dynamic `/profile` cards.
- **Caching:** In-memory `Map` (or Redis) for active combat sessions and cooldowns.

2. Directory Structure

The AI should initialize the project with the following domain-driven structure:

```

cartethyia/
├── src/
│   ├── assets/           # Fonts, default backgrounds for Canvas
│   ├── commands/        # Slash commands (social, rpg, system)
│   ├── components/      # Discord interactive components (buttons, modals)
│   ├── core/
│   │   ├── ai/          # Ollama API wrapper & prompt queue
│   │   ├── combat/      # Turn-based combat engine & session state
│   │   └── handlers/    # Dynamic command/event loaders
│   ├── database/        # Prisma client and helper functions
│   ├── events/          # Discord.js event listeners (ready, interactionCreat
│   ├── rpg/             # Game logic (Echo rolling, damage math, elements)
│   ├── utils/           # Embed builders, color extractors, RNG helpers
│   └── index.ts         # Bot entry point
├── prisma/
│   └── schema.prisma    # Database schema
└── .env                # Tokens and DB URLs

```



```

    isMain      Boolean    @default(true) // If true, dictates ultimate ability

    mainStat    String     // JSON string or specific column (e.g., {"type": "CritRat
    subStats    String     // JSON string tracking up to 5 stats and if they are Stas
}

model Interaction {
    id          String     @id @default(cuid())
    userId      String
    userBId     String
    affinity     Int       @default(0) // Levels up intimacy between these two specif

    sender      User       @relation("Sender", fields: [userId], references: [id])
    receiver     User       @relation("Receiver", fields: [userBId], references: [id])

    @@unique([userId, userBId])
}

```

4. The Vibe Engine (Social & Local AI Integration)

Socializing generates resources. AI generates the narrative.

4.1 Interaction Flow & Loot

1. User A types `/interact pat @User B`.
2. Check User A's Resonance Capacity. If > 0, deduct 1.
3. Bot queries DB for `Interaction` between User A and User B to get **Affinity Level**.
4. Generate AI Flavor text based on Affinity.
5. Drop Loot based on interaction type (Aggressive = Weapon Ores; Affectionate = Echo Modules).
6. Render Embed with Anime GIF + AI Text. Include a `[Return Pat]` Discord Button.
7. If User B clicks `[Return Pat]`, both users get 2x Loot Multiplier.

4.2 Local AI Architecture (Ollama)

- **Hardware constraint:** Running on RTX 5050. Cannot handle concurrent requests.
- **Implementation:** Must use a queue (e.g., `p-queue` set to `concurrency: 1`).
- **Latency Handling:** The bot *MUST* call `await interaction.deferReply()` instantly to prevent Discord API timeouts while the local GPU generates the text.
- **Prompt Scaling:** The prompt sent to Ollama must dynamically change based on DB `affinity`:
 - *Affinity 0-10:* "Strangers. Awkward, polite."
 - *Affinity 11-50:* "Casual friends. Playful."
 - *Affinity 51-100:* "Bonded. Warm, affectionate."
 - *Affinity 101+:* "Soul-linked/Married. Deeply devoted, romantic."

5. RPG Mechanics & Progression

5.1 Leveling & World Level (WL)

- **Chat EXP:** Every message sent grants basic Character EXP.
- **WL Gates:** Character level caps at 20 (WL0), 40 (WL1), 60 (WL2), etc.
- **Ascension Boss:** To break a cap, users must solo an `/ascension` boss. Winning permanently increases World Level, which removes low-tier Echo drops (At Max WL, every drop is 5-Star).

5.2 Elements & Resonance (Unlocked at WL 1)

- **Fusion (Fire):** +Crit Rate. (Burst/DoT).
- **Glacio (Ice):** +Defense. (Crowd control/Delay).
- **Electro (Lightning):** +Energy Regen. (Fast Ultimates).
- **Aero (Wind):** +Speed. (Turn order manipulation).
- **Havoc (Dark):** +Lifesteal. (Solo survival).
- **Spectro (Light):** +Healing. (Co-op support).

5.3 The Paradox System (Endgame Gear)

Echoes have 5 substats unlocked at levels 5, 10, 15, 20, 25 (using *Tuning Modules* and *Sealing Tubes*).

- **Paradox Cores:** Extremely rare drop. Rerolls all unlocked substats simultaneously.
- **Stasis Locks:** Premium/Insane Grind item. Locks a single perfect substat (e.g., Crit DMG 20%) so it doesn't change when a Paradox Core is used. Cost scales exponentially per locked stat.
- **Synthesis:** Burn 5 trash 5-Star Echoes to generate 1 new 5-Star Echo.

6. Combat Engine (Turn-Based QTE & Synergy)

Combat is handled via Discord UI buttons and embeds, processing state entirely in-memory during the fight to save DB reads/writes.

6.1 Fight Loop

1. **Turn Order:** Decided by the `Speed` stat.
2. **Player Turn UI:** Show buttons: `[Basic Attack]` , `[Resonance Skill]` , `[Ultimate (Requires 100 Energy)]` .
3. **The Stagger Bar (Vibration):** Bosses have a stagger meter. Hitting elemental weaknesses or Parrying depletes it. When broken, the boss is "Shattered" (Skips 2 turns, takes 2.5x damage, all hits Crit).
4. **Co-op Concerto:** If fighting with others, players can `[Pass Turn / Tag Out]` to give the next player a massive "Intro Skill" multiplier.
5. **Elemental Reactions:** (e.g., Glacio followed by Fusion triggers `Melt` = 2.0x combined damage).
6. **Boss Turn:** Boss attacks. Damage math is calculated and applied to the player.

Users can run `/dispatch` to send their character out for 4, 8, or 12 hours. They cannot raid during this time. Automatically yields EXP and materials upon return.

7. AI IDE Implementation Roadmap

Instruct your AI IDE to build the bot in the following order:

1. **Phase 1: Foundation.** Initialize Discord.js client, load `.env`, setup the modular command and event handlers. Establish Prisma connection and push the schema.
2. **Phase 2: The Local AI Engine.** Setup the `p-queue` service that connects to `http://localhost:11434/api/generate`. Build the helper function that constructs Intimacy-scaled prompts.
3. **Phase 3: The Social Loop.** Build `/interact pat` as a proof-of-concept. Fetch affinity, generate AI text, attach a placeholder GIF, and add the `[Return]` button. Test the resource drop logic.
4. **Phase 4: RPG Profiles.** Build the `/profile` command using Canvas to generate a beautiful stat card (Avatar, Level, WL, Element, Weapon).
5. **Phase 5: Echo Management.** Build the UI commands (`/echo inventory` , `/echo upgrade`) utilizing Select Menus. Implement the Paradox Core reroll logic.
6. **Phase 6: Combat System.** Build the in-memory combat session class. Create the `/ascension` boss logic first as a solo baseline, then expand to Co-op Calamity Raids.